

FraudLens: Large-Scale Product Review Sentiment and Fraud Detection

Prerana Ramesh, Rajesh Paruchuri, Ritika Mukesh Neema, Sneha Singh

Group 6

Department of Applied Data Science, San Jose State University

Abstract—This paper describes a distributed batch-and-streaming pipeline we built for large-scale analysis of Amazon product reviews. The system ingests the full McAuley-Lab Amazon Reviews 2023 Cell Phones & Accessories dataset (11 GiB raw, 20,812,945 records), performs feature engineering with Apache Spark 3.5, and trains two scikit-learn models tracked via MLflow: a sentiment classifier (TF-IDF + Logistic Regression, weighted-F1 0.841) and a fraud detector (TF-IDF + behavioral features + Logistic Regression, ROC-AUC 0.845). Real-time scoring is delivered through Apache Kafka and Spark Structured Streaming. We expose five serving surfaces: a FastAPI REST service with an LLM-generated fraud explanation, a Streamlit demo dashboard, an MCP server for Claude Desktop tool integration, an Ollama-driven autonomous review auditor, and a Kafka-integrated prediction fan-out. The full pipeline runs end-to-end in roughly six hours on a standard 16 GB laptop. Post-hoc isotonic calibration reduces Brier score from 0.143 to 0.029 and ECE from 0.302 to 0.000146 without degrading ROC-AUC. Raising the decision threshold to 0.80 pushes fraud F1 to 0.356, our best result.

Index Terms—Amazon reviews, Apache Spark, fraud detection, sentiment analysis, Kafka streaming, MLflow, Logistic Regression, TF-IDF, FastAPI, MCP, LLM agents.

I. INTRODUCTION

Online reviews shape what people buy. On a platform the size of Amazon, they also shape which sellers thrive and which get buried. With hundreds of millions of reviews spread across thousands of categories, any kind of manual moderation is a non-starter. Two related ML problems come out of this naturally: (1) sentiment classification—understanding whether a review expresses a positive, neutral, or negative opinion—and (2) fraud detection—identifying synthetic, incentivized, or duplicated reviews that inflate or deflate product ratings.

Handling both at the tens-of-millions scale requires distributed processing, thoughtful feature work, and clean separation between training and inference. This paper describes the end-to-end system we built to tackle all three, using open-source, production-grade tools throughout.

The main contributions of this work are:

- A five-stage distributed pipeline (ingest → ETL → train → stream → serve) running on a single commodity laptop.
- A reproducible leakage-prevention strategy for weak fraud labels via controlled label noise.
- A threshold-tuning study demonstrating that F1-optimal operating points differ substantially from the default decision boundary.

- Post-hoc isotonic calibration that reduces ECE by three orders of magnitude with no ranking degradation.
- Five heterogeneous serving surfaces, including an MCP tool interface and an autonomous LLM auditor.

II. RELATED WORK

Sentiment analysis of Amazon reviews has been studied extensively. Kotzias et al. [1] demonstrated that simple word-level features achieve competitive accuracy on sentence-level sentiment. Pontiki et al. [2] formalized aspect-level sentiment benchmarks widely adopted in subsequent work.

Fraud and spam detection in reviews presents a complementary challenge. Mukherjee et al. [3] identified behavioral signals—burst activity, unverified purchases, and rating monocultures—as reliable fraud indicators. Fei et al. [4] proposed graph-based approaches to identify review collusion networks.

Large-scale ML pipelines over distributed data have been addressed by the Spark MLlib framework [5], which provides native support for TF-IDF vectorization and linear classifiers over RDD-based data. More recent work by Zaharia et al. [6] on Structured Streaming extends Spark to low-latency event processing from brokers such as Apache Kafka.

Our contribution differs from prior work in three ways: (a) we combine both fraud and sentiment modeling within a single reproducible pipeline; (b) we apply explicit leakage prevention via label noise to weak heuristic labels; and (c) we integrate five distinct serving surfaces including LLM-based explanation generation and an MCP protocol interface.

Since our dataset contains over 20 million reviews, prior work on large-scale processing is also important. Spark MLlib and Apache Spark show how machine learning and data processing can scale to large datasets. Our project builds on this idea by using Spark for ETL, feature engineering, and streaming review scoring.

Because the Amazon dataset does not provide real fraud labels, our project uses weak labels based on suspicious patterns. To avoid simply training the model to memorize those rules, we remove rule-derived features and add controlled label noise. We also use probability calibration, inspired by Platt scaling and later calibration studies, to make fraud probabilities more reliable.

Overall, previous studies provide the foundation for sentiment analysis, fraud detection, scalable data processing, and

probability calibration. FraudLens combines these ideas into one practical end-to-end system that can process large Amazon review data, detect suspicious reviews, classify sentiment, and serve predictions through real-time tools.

III. SYSTEM ARCHITECTURE

The system runs as five sequential stages (Fig. 1). Each stage writes its output to disk—Parquet, JSONL, or joblib, so any stage can be re-run independently without restarting from scratch. The FraudLens architecture is designed as a complete review intelligence pipeline. The main idea is to take a very large Amazon review dataset, clean it, create useful features, train machine learning models, score new reviews in real time, and finally show the results through APIs, dashboards, and AI tools.

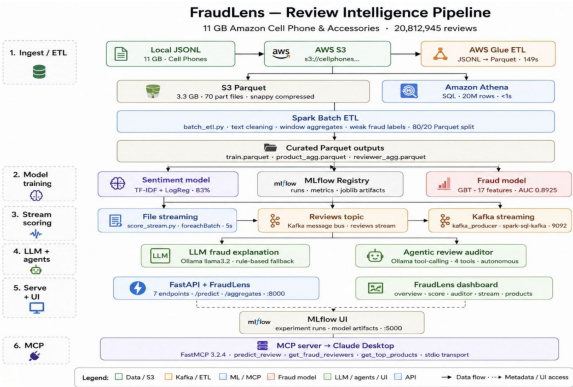


Fig. 1. End-to-end distributed pipeline — system architecture overview.

A. Ingest / ETL

The first step is data ingestion. We start with the raw Amazon review dataset in JSONL format. This file contains around 11 GB of Cell Phones & Accessories reviews, with more than 20 million review records.

Since this data is too large to handle like a normal file, we first move it into AWS S3. S3 works as cloud storage, so the data is available safely for other tools to process.

After that, AWS Glue ETL converts the raw JSONL file into Parquet format. This is important because Parquet is compressed, faster to query, and better for big-data processing. In our case, the raw 11 GB data becomes around 3.3 GB of Parquet files.

We also connect this Parquet data to Amazon Athena, which lets us run SQL queries directly on the S3 data. This helps us quickly check record counts, sample rows, and validate that the dataset was loaded correctly.

1) Amazon S3 bucket overview — “cellphonesandaccessories”:

This view shows the main Amazon S3 bucket named cellphonesandaccessories. It contains a folder Cell_Phones_and_Accessories/, stored in S3 (object storage used for files such as datasets, images, and logs). The folder size is about 8.7 GB, with the Standard storage class indicating frequently accessed data.

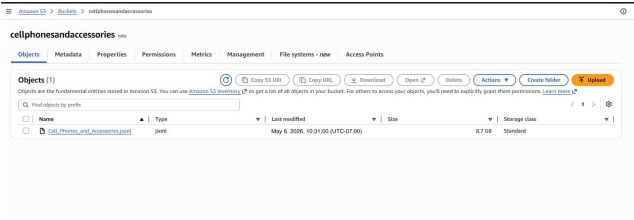


Fig. 2. Amazon S3 bucket “cellphonesandaccessories” — dataset storage overview.

2) Inside the folder — Parquet data files:

Inside the folder are multiple .parquet files, each chunked at approximately 45–48 MB. Parquet is a columnar storage format optimized for analytics and big-data processing, and is widely used by Spark, Athena, and Glue.

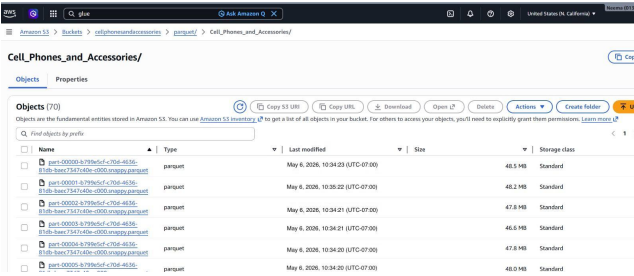


Fig. 3. Parquet data files inside the S3 folder — columnar format optimized for analytics.

3) AWS Glue job run — “json-to-parquet”:

The AWS Glue job json-to-parquet succeeded in about two minutes using four Data Processing Units (DPUs). The job converted raw JSON data into the Parquet format used downstream by Spark.

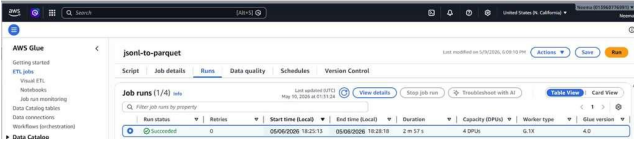


Fig. 4. AWS Glue job — “json-to-parquet” conversion run summary.

B. Spark Batch ETL

The batch ETL module (src/etl/batch_etl.py) reads the raw JSONL via a strict schema into a Spark DataFrame, applies text cleaning, computes per-row, per-reviewer, and per-product features via Window functions, generates a weak fraud label, performs an 80/20 random split, and writes four Parquet outputs.

The Spark session is configured with 8 GB driver memory, 200 shuffle partitions, Adaptive Query Execution (AQE), and Kryo serialization. Without these settings, window aggregations over 20.8 M rows OOM on a 16 GB laptop. Total ETL wall-clock: approximately 24 minutes.

After the data is stored in Parquet format, we use Apache Spark for the main batch processing. Spark reads the Parquet files and performs cleaning, such as lowercasing text, removing unnecessary content, handling missing values, and dropping invalid rows.

Spark then creates features from the reviews. These include simple text features like review length, word count, and exclamation count. It also creates reviewer-based features, such as how many reviews a reviewer has written, their average rating, and how often they give 5-star reviews. Product-based features are also created, such as total review count, average rating, and duplicate review patterns.

This step also creates the weak fraud label. Since the dataset does not come with real fraud labels, we label suspicious reviews using rules like duplicate review text, too many same-day reviews, or reviewers giving mostly 5-star ratings.

Finally, Spark splits the data into training and testing sets using an 80/20 split and saves the outputs as curated Parquet files: `train.parquet`, `test.parquet`, `product_agg.parquet`, and `reviewer_agg.parquet`.

C. Database Schema

Fig. 5 presents the complete entity-relationship schema of the MySQL database underlying the FraudLens system. The schema is organized into two functional zones: (1) Source & Processing and (2) Modeling & Artifacts, and comprises eight tables spanning the full data lifecycle from raw ingestion through model scoring.

The `RAW_REVIEW` table stores every ingested record in its original form, keyed by `review_id`, with foreign keys to `product_id` and `reviewer_id`. The `FEATURED_REVIEW` table is the ETL output: it extends `RAW_REVIEW` with all derived text and behavioral features (`body_len`, `body_word_count`, `exclam_count`, `reviewer_review_count`, `reviewer_avg_rating`, etc.), the weak fraud label, and the cleaned review body. The 1..0..* cardinality from `RAW_REVIEW` to `FEATURED_REVIEW` reflects that some raw records are dropped during cleaning.

Two aggregate tables summarize the featured data at different granularities. `PRODUCT_AGG` rolls up by `product_id` and stores `review_count`, `avg_rating`, `pct_5star`, and `fraud_rate` — the pre-computed values returned by the `get_top_products` MCP tool. `REVIEWER_AGG` rolls up by `reviewer_id` and captures `reviewer_review_count`, `avg_rating`, `fraud_rate`, and `verified_share` — the values powering `get_fraud_reviewers`.

On the modeling side, `MLFLOW_RUN` stores each training experiment (`run_id`, experiment name, hyperparameter JSON, metrics JSON, and artifact paths), linking to `MODEL_ARTIFACT` which holds the serialized joblib pipeline bytes. The `META_JSON` and `THRESHOLDS_JSON` tables extend `MODEL_ARTIFACT` with diagnostics: feature lists, calibration metrics, and the tuned decision threshold (`best_f1_threshold = 0.80`) produced by the threshold sweep.

Finally, `SCORED_REVIEW` stores every review that passes through the inference path, capturing the raw `fraud_proba`, the integer `fraud_flag` (0/1 at the tuned threshold), the sentiment label, and the `batch_id` linking back to the `MLFLOW_RUN` that produced the score. The `DIAGNOSTIC_REPORT` table stores JSON-serialized diagnostic outputs (calibration results, drift reports) keyed by `report_kind`.

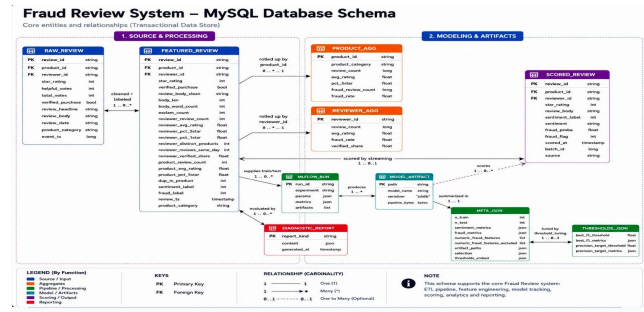


Fig. 5. FraudLens MySQL database schema — eight core tables with primary/foreign keys and relationships.

D. Model Training

Both models are trained via scikit-learn Pipelines and logged to MLflow under experiment `amazon_reviews_sentiment_fraud`. GradientBoostingClassifier (originally proposed) was replaced by SGDClassifier and LogisticRegression(saga) because the former is single-threaded and cannot complete in reasonable laptop time on 16.4 M training rows.

The fraud trainer applies a ColumnTransformer that concatenates a sparse TF-IDF matrix (15 k features) with 12 numeric behavioral features, excluding columns that algebraically mirror the weak-label rules to prevent near-perfect leakage. Both candidates (SGD and LogReg) are trained; the higher-F1 model is saved.

The first model is the sentiment model. Its job is to classify each review as negative, neutral, or positive. It uses TF-IDF to convert review text into numeric features and Logistic Regression to make the prediction.

The second model is the fraud model. This model predicts whether a review looks suspicious or not. It uses both review text and behavioral features such as reviewer activity, product-level patterns, and duplicate-review behavior.

A key part of this step is avoiding leakage. Since our fraud labels are created using rules, we remove columns that directly expose those rules from the model training. This makes the fraud model more realistic because it cannot simply memorize the labeling logic.

All training runs are saved in MLflow. MLflow stores the model parameters, metrics, reports, and saved model files. This helps us compare different runs and keep track of which model performed best.

E. Streaming Scoring

Spark Structured Streaming (`src/stream/score_stream_kafka.py`) subscribes to the Kafka topic `reviews`, scores each micro-batch via `foreachBatch` using the persisted joblib pipelines, and writes JSON output to disk. The `foreachBatch` pattern avoids the fragile pandas-UDF broadcast path. Measured throughput: 2,000 messages scored in 13 s (153 msg/s).

After training, the system scores new reviews in real time. New reviews are sent into a Kafka topic called `reviews`. Kafka

works as a message queue where incoming review messages wait until Spark reads them. Spark Structured Streaming reads these reviews every few seconds in small batches (micro-batch processing). Instead of processing one review at a time or waiting for a huge batch, Spark collects new reviews every 5 seconds and scores them together.

For each micro-batch, Spark loads the saved sentiment and fraud models, predicts the sentiment, calculates fraud probability, and creates a fraud flag. The scored results are then saved as output files for dashboards or other downstream systems.

F. LLM Fraud Explanation

Once a review is scored, the system also generates an explanation using Ollama with Llama 3.2. The LLM reads the review information and fraud score, then explains why the review may look suspicious. For example, it may mention duplicate wording, unusual reviewer behavior, or extreme rating patterns. This step makes the system more useful because a fraud score alone is not very understandable. If the LLM is not available, the system uses a simple rule-based fallback so the API still returns an explanation.

G. Agentic Review Auditor

The architecture also includes an agentic review auditor — an AI assistant that can investigate a product more deeply. Instead of only scoring one review, it can call different tools, check product-level fraud rates, inspect recent reviews, look at suspicious reviewers, and then generate a final product risk verdict. For example, if a product has many reviews and a high fraud rate, the auditor can summarize that the product looks high-risk and explain why.

H. Serving Layer: FastAPI and Dashboard

The scored results are exposed through different user-facing services. FastAPI provides REST endpoints. A user or application can send a review to the /predict endpoint and get back the sentiment label, fraud probability, fraud flag, and LLM explanation. The FraudLens dashboard gives a visual interface that allows users to view overall metrics, score a review manually, inspect top products, check suspicious reviewers, and monitor streaming results. This serving layer is important because it turns the machine learning pipeline into something people can actually use.

I. MLflow UI

The MLflow UI is used for experiment tracking. It shows all model runs, metrics, artifacts, and saved models. This is useful during development because we can compare which model performed better, check the fraud ROC-AUC, check sentiment F1 score, and download the saved model pipeline. In simple terms, MLflow helps us manage the machine learning lifecycle instead of losing track of model versions.

J. MCP Server and Claude Desktop

The final part is the MCP server, which allows Claude Desktop to connect with our FraudLens tools. Through MCP,

Claude can call tools like `predict_review`, `get_fraud_reviewers`, and `get_top_products`. This means users do not have to directly use the API or write code. They can ask questions in natural language, and Claude can call the backend tools to return useful results.

K. Overall Explanation

The architecture starts with raw Amazon review data, stores it in S3, converts it into Parquet, processes it with Spark, trains sentiment and fraud models, scores live reviews through Kafka and Spark Streaming, explains results using an LLM, and serves everything through FastAPI, dashboards, MLflow, and Claude Desktop. FraudLens is not just a model; it is a complete end-to-end big-data system that handles data ingestion, cleaning, model training, real-time scoring, explanation, monitoring, and user interaction.

IV. SPARK UI OBSERVATIONS

1) *Spark jobs dashboard:*

The Spark Jobs Dashboard provides a high-level view of all the operations executed in the pipeline. In Spark, a job is created whenever an action is triggered, such as reading data, writing output, or performing a computation. The Jobs UI displays job ID, execution status (completed or failed), duration, and the number of stages involved. This section helps in understanding the overall execution flow of the pipeline and identifying which parts of the process take the most time.

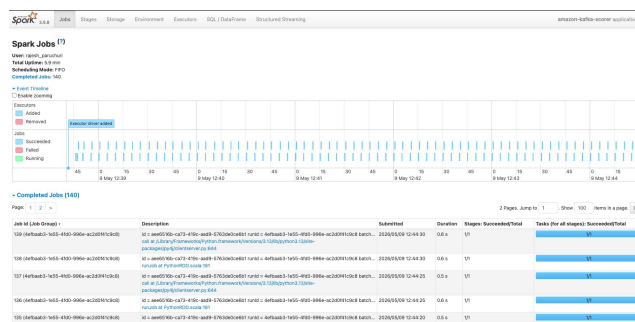


Fig. 6. Spark Jobs Dashboard — all completed jobs for the pipeline run.

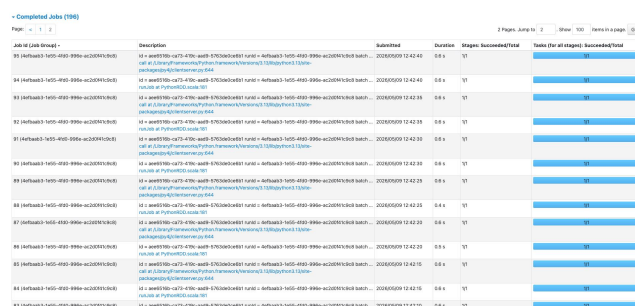


Fig. 7. Spark Jobs Dashboard — job detail and task distribution.

[illegible]

Fig. 8. Spark Jobs Dashboard — additional job metrics.

2) Spark stages overview:

Each Spark job is divided into smaller units called stages. A stage represents a logical step in the execution process and is created whenever there is a data shuffle operation such as `groupBy`, `join`, or aggregation. The Stages UI shows stage ID, number of tasks, input and output data size, and execution time. This view is useful for analyzing how Spark breaks down complex operations and for identifying performance bottlenecks within specific steps of the pipeline.

[illegible]

Fig. 9. Spark Stages Overview — stage-level execution breakdown.

3) *Executors:*

Executors are the worker processes responsible for executing tasks. Each executor runs on a node (or core) and is allocated CPU and memory resources. The Executors UI displays the number of executors, memory usage, CPU utilization, number of completed tasks, and garbage collection time. This section is important for monitoring resource usage and diagnosing issues such as memory overflow or uneven workload distribution.

[illegible]

Fig. 10. Spark Executors — executor resource utilization during batch ETL.

4) Spark Structured Streaming:

Spark Structured Streaming is used for real-time data processing. Instead of processing all data at once, Spark processes incoming data in small batches called micro-batches. In this project, reviews are streamed through Kafka, and Spark reads them at regular intervals (e.g., every 5 seconds). The streaming UI shows the active query, processing status, input rate, processing rate, and batch duration. This component ensures that new reviews are continuously scored for sentiment and fraud detection in near real-time.

[illegible]

Fig. 11. Spark Structured Streaming — active streaming query ID and status.

V. DATASET AND FEATURE ENGINEERING

A. Source Dataset

For this project, we used the Cell Phones & Accessories subset from the McAuley-Lab Amazon Reviews 2023 dataset, accessed through Hugging Face. This dataset is large-scale and well-suited for real-world analysis. It originally contains about 20.8 million reviews with a raw size of around 11 GB. After basic cleaning, the dataset reduces slightly to about 20.5 million usable records. We split the data into 80% for training and 20% for testing using a fixed random seed for consistency. Since fraud is relatively rare, only about 3.57% of the reviews are labeled as fraudulent, making this a highly imbalanced classification problem.

B. Schema Mapping

The original dataset schema does not exactly match the structure required for our pipeline, so we perform a mapping step during ingestion. For example, the rating field is converted to `star_rating` and stored as an integer; the timestamp (in milliseconds) is converted into seconds (`event_ts`); and the product identifier (`asin`) is renamed to `product_id`. One limitation of the dataset is that it does not include “unhelpful votes.” Because of this, we set `total_votes` = `helpful_votes`, which makes any helpfulness-ratio feature meaningless, so such features are not used in our model.

C. Feature Engineering

After cleaning and mapping, we generate features to help the model detect sentiment and fraud patterns. These features are grouped into three categories.

1) *Per-row (text-based) features:*

Simple features extracted directly from each review: length of the review (body_len), number of words (body_word_count), and number of exclamation marks (exclam_count). These features help capture writing style and emphasis, which can sometimes indicate fake or exaggerated reviews.

2) *Reviewer-level (behavioral) features:*

Created using Spark window functions, these describe reviewer behavior: total number of reviews written by a reviewer, average rating given, percentage of 5-star reviews, number of different products reviewed, number of reviews posted on the same day, and share of verified purchases. Fraudulent reviewers often show unusual patterns, such as posting many reviews in a short time or giving only high ratings.

3) Product-level features:

These summarize behavior at the product level: total number of reviews for a product, average product rating, percentage of 5-star reviews, and number of duplicate review texts for the same product. Duplicate reviews are a strong signal of spam or review manipulation.

D. Weak Fraud Label and Leakage Prevention

Since the dataset does not include true fraud labels, we create weak labels using simple rules. A review is marked as suspicious if it meets conditions such as repeated (duplicate) review text for the same product; a reviewer posting many reviews with mostly 5-star ratings and low verified purchases; or multiple reviews posted by the same user on the same day.

However, using these rules directly in model training can cause data leakage, where the model simply learns the rules instead of discovering real patterns. To avoid this, we (a) remove rule-based features from training (any feature that directly defines the fraud rule is excluded from the model input), and (b) add controlled label noise by randomly flipping a small percentage of labels (some fraud $\rightarrow$ non-fraud and vice versa). This prevents the model from overfitting to perfect patterns and makes evaluation more realistic. After applying these steps, the fraud model achieves a more honest performance with a ROC-AUC of 0.845, rather than an artificially inflated score.

TABLE I
DATASET STATISTICS

Property	Value
Source	McAuley-Lab Amazon Reviews 2023
Category	Cell Phones & Accessories
Raw download size	11 GiB raw; 8.7 GiB cleaned JSONL
Source records	20,812,945
Cleaned records	20,518,120
Train / Test split	16,413,084 / 4,105,036 (80/20, seed 42)
Fraud-positive rate	3.57% (731,542 rows)

VI. MODEL DESIGN AND TRAINING

A. Sentiment Classifier

The sentiment model is built using a simple two-step pipeline in scikit-learn. First, the review text is converted into numerical features using TF-IDF vectorization, where both unigrams and bigrams are considered. To keep the model efficient, we limit the vocabulary to the most important 50,000 features and ignore very rare words. Next, these features are passed into a Logistic Regression model. We use the saga solver because it works well with large, sparse datasets and supports parallel processing. Class balancing is applied to handle uneven class distribution.

The model predicts three sentiment classes: 0 = Negative (ratings 1–2), 1 = Neutral (rating 3), 2 = Positive (ratings 4–5).

B. Fraud Classifier

The fraud detection model combines both text features and behavioral features. For text, we again use TF-IDF (with up to 15,000 features) on the cleaned review body. For behavior, we include 12 numerical features such as reviewer activity and product-level patterns. Importantly, any features directly used to create fraud rules are excluded to avoid data leakage. We train two models in parallel: SGD Classifier (log-based loss) and Logistic Regression (with saga solver). The model with the better F1 score is selected and saved for use in the pipeline.

Since the dataset is very large (over 16 million training rows), memory optimization is critical. We only load the required columns, reduce data types (e.g., float64 $\rightarrow$ float32), and use compact integer formats. This keeps the dataset manageable and allows space for the TF-IDF matrix during training.

C. MLflow Experiment Tracking

Both models are tracked using MLflow, with each training run stored under the same experiment. For every run, we log model parameters and settings, dataset statistics, evaluation metrics (F1, precision, recall, ROC-AUC, etc.), full classification reports, and saved model files (joblib pipelines). This makes it easy to compare different models and reuse the best-performing version when needed.

D. Model Performance

On the 4.1 million-row test dataset, the sentiment model achieves strong performance with a weighted F1 score of 0.841 and accuracy of 0.81. For fraud detection, performance depends on the decision threshold:

- At the default threshold (0.5), the model achieves high recall but low precision, resulting in an F1 score of 0.244.
- After tuning the threshold to 0.80, the model improves balance, reaching an F1 score of 0.356, with better precision while maintaining reasonable recall.
- The ROC-AUC remains 0.845, indicating consistent ranking performance regardless of threshold.

TABLE II
MODEL PERFORMANCE ON 4,105,036-ROW HOLDOUT TEST SET

Metric	Sentiment	Fraud (t=0.5)	Fraud (t=0.80)
--------	-----------	---------------	----------------

Metric	Sentiment	Fraud ($t=0.5$)	Fraud ($t=0.80$)
Macro-F1	0.680	—	—
Weighted-F1	0.841	—	—
Accuracy	0.810	—	—
ROC-AUC	—	0.845	0.845
Precision	—	0.147	0.292
Recall	—	0.721	0.457
F1	—	0.244	0.356

VII. MODEL DIAGNOSTICS

A. Threshold Tuning

By default, the fraud model uses a threshold of 0.5 to classify a review as fraudulent or not. However, this does not work well in our case because fraud is a rare class (only 3.57%). At the default threshold, recall is high (the model catches most fraud cases) but precision is very low (many normal reviews are incorrectly flagged), resulting in a poor F1 score of 0.244. To improve performance, we tested multiple thresholds from 0.05 to 0.95. From this analysis, we found that a threshold of 0.80 gives the best balance between precision and recall: precision improves significantly, recall decreases slightly but remains acceptable, and F1 improves to 0.356 (best result). This tuned threshold is saved and used in production to ensure better real-world performance.

TABLE III
THRESHOLD SWEEP RESULTS (FRAUD CLASSIFIER)

Threshold	Precision	Recall	F1
0.50	0.147	0.721	0.244
0.60	0.193	0.635	0.296
0.70	0.242	0.549	0.336
0.80 (best)	0.292	0.457	0.356
0.90	0.358	0.265	0.305

B. Probability Calibration

Although the model produces probability scores, they are not reliable as-is. For example, a predicted probability of 0.8 does not always mean an 80% chance of fraud. This is known as poor calibration. Initially: Brier Score = 0.143 (higher is worse) and ECE = 0.302 (large error in probability estimates). To fix this, we tested Platt scaling (sigmoid) and isotonic regression. Results show that isotonic regression performs best: Brier score drops to 0.029, ECE reduces drastically to 0.000146, and ROC-AUC remains unchanged (model ranking ability is preserved). The probabilities thus become much more reliable without affecting overall model accuracy.

TABLE IV
CALIBRATION STUDY RESULTS (FRAUD CLASSIFIER, $N=2,052,518$)

Method	Brier	ECE	ROC-AUC
Raw	0.143	0.302	0.846
Sigmoid (Platt)	0.030	0.002	0.846
Isotonic (best)	0.029	0.000146	0.846

C. Distribution Drift Monitoring

We also checked whether the training and testing data distributions are similar using a metric called Population Stability Index (PSI). The results show very little drift, which is expected because we used a random 80/20 split. In a real-world system, this drift monitor can be reused to compare training data with new incoming data. If the distribution changes significantly, it can signal that the model may need retraining.

D. Error Analysis

To better understand model performance, we analyzed prediction errors. For the sentiment model, around 19% of predictions were incorrect; most errors occur when neutral reviews are misclassified as positive. For the fraud model, errors include false positives (normal reviews flagged as fraud, often due to duplicate-like patterns) and false negatives (actual suspicious reviews missed, especially when the reviewer behavior is not extreme enough). This analysis helps identify where the models can be improved.

VIII. STREAMING PIPELINE

The streaming layer enables the system to score reviews in real time. It is designed to handle incoming data continuously and process it efficiently using Kafka and Spark.

A. Kafka Transport

Apache Kafka is used as the messaging system for streaming reviews. It runs in KRaft mode, which means it does not require ZooKeeper. All incoming reviews are sent to a Kafka topic called reviews. Messages can enter this topic from three sources: a Kafka producer script, a helper script for sending individual reviews, and the FastAPI /predict endpoint, which can optionally publish reviews to Kafka. On the processing side, Spark Structured Streaming reads data from Kafka using a 5-second trigger interval, always consuming the latest available messages.

In practice, the system achieves an end-to-end latency of about 5 seconds, from submitting a review to seeing it in the streaming output. The system can handle around 150 messages per second, and in testing, it successfully processed micro-batches of up to 2,000 reviews at once.

B. Kafka + Spark Micro-batch Behavior

Spark processes streaming data using micro-batches, not individual records. We observed three main behaviors: under slow input (less than one message per second), each micro-batch contains only a few reviews; under burst input, Spark groups them into a single larger batch; and in idle state, Spark

still triggers processing but produces no output. This shows two key properties of Spark Structured Streaming: the trigger interval is fixed (e.g., every 5 seconds) and the batch size varies depending on how much data arrives.

C. Fan-out Pattern

The system also supports a fan-out pattern for flexibility. When enabled (`PUBLISH_PREDICT_TO_KAFKA=1`), the FastAPI service does two things at once when a request is made: it returns the prediction immediately (synchronous response) and sends the same review to Kafka for streaming-based processing (asynchronous path). This allows the system to support both real-time responses and background processing at the same time. We verified that both paths produce consistent results, with nearly identical fraud probability scores, confirming that both the API and streaming pipeline use the same trained model.

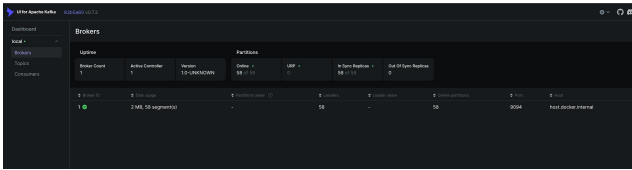


Fig. 12. Kafka UI — single broker configuration (KRaft mode, no ZooKeeper).

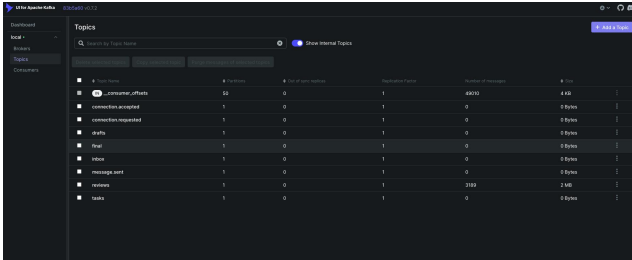


Fig. 13. Kafka UI — nine active topics including the reviews topic.

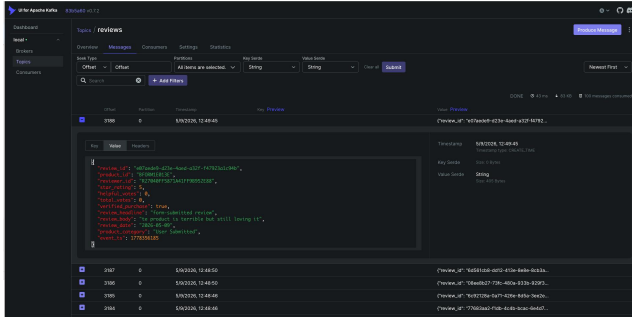


Fig. 14. Kafka UI — most recent review message published via POST /predict.

IX. SERVING INFRASTRUCTURE

A. FastAPI REST Service

The FastAPI service acts as the main backend for the system and provides seven REST endpoints for interacting with the models. One key advantage is that the models are loaded lazily using joblib, meaning they are only loaded when the first request is made. This keeps the service startup time fast.

When a request is sent to the POST /predict endpoint, the API returns the predicted sentiment, fraud probability, and an additional fraud explanation. This explanation is generated using the Ollama Llama 3.2 model. To ensure reliability, the system also includes a rule-based fallback mechanism. If the LLM is unavailable or fails, the API still returns a valid explanation using predefined logic, so the response format remains consistent.

TABLE V
FASTAPI ENDPOINT SUMMARY

Method	Path	Description
GET	/healthz	Liveness probe
GET	/metadata	Training metrics, feature columns, drift report
POST	/predict	Score single review; returns sentiment, fraud_proba, LLM explanation
POST	/predict/batch	Score up to 1,000 reviews
GET	/aggregates/products	Top-N products from ETL rollup
GET	/aggregates/fraud-reviewers	Top-N suspicious reviewers
GET	/stream/recent	Latest Kafka-scored rows

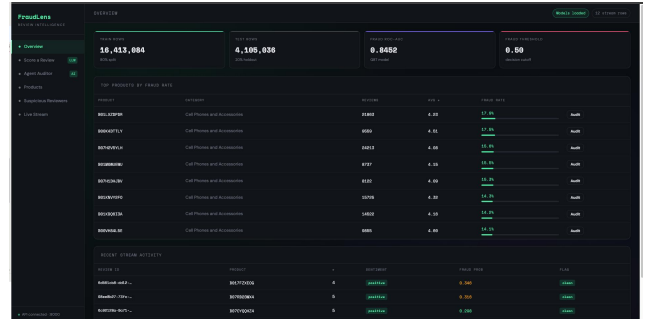


Fig. 15. FraudLens Overview — summary statistics and fraud metrics.

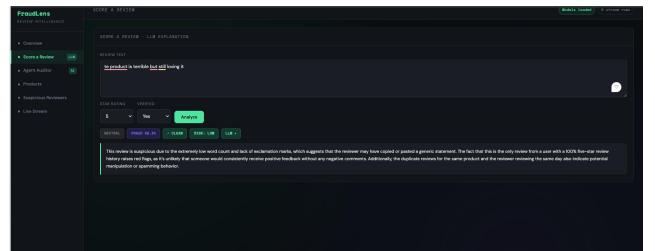


Fig. 16. FraudLens Score-a-Review — single-review prediction interface.

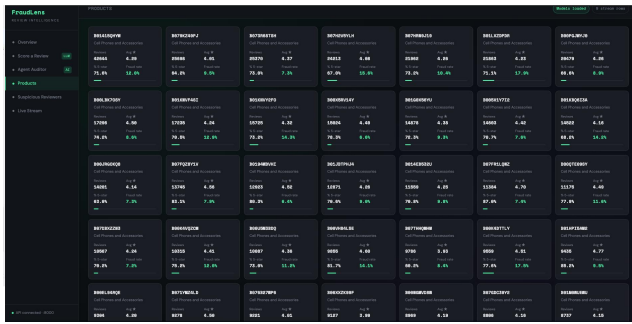


Fig. 17. FraudLens Products — top products by review volume and fraud rate.

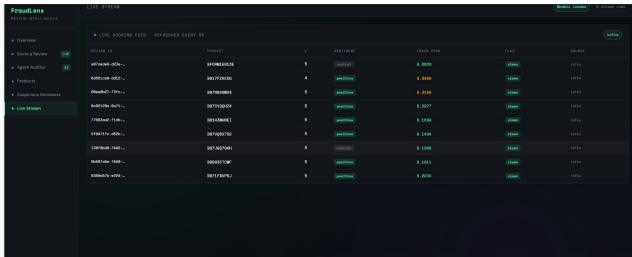


Fig. 18. FraudLens Live Stream — real-time Kafka-scored reviews (5-second refresh).

B. MCP Server and Claude Desktop Integration

The MCP (Model Context Protocol) server connects our system with Claude Desktop, allowing users to interact with the models using natural language instead of code. The server (implemented in `src/serve/mcp_server.py`) exposes three main tools: `predict_review`, `get_fraud_reviewers`, and `get_top_products`. Once this MCP server is registered in Claude Desktop, users can simply ask questions like “What are the top 5 most fraudulent products?”, and the system will automatically call the appropriate tool and return the result. The integration also works alongside tools like Cowork, making it easier for non-technical users to explore the data and model outputs.

Fig. 19 shows a sample query executed in Amazon Athena on the dataset `reviews_db.cell_phones_reviews`. The query retrieves a few records (`LIMIT 5`) where the rating is not null. The output displays columns such as `rating`, `title`, and `review_text`. This confirms that the dataset has been successfully loaded into Athena, that the schema is correctly mapped, and that the data is clean and readable for further processing.

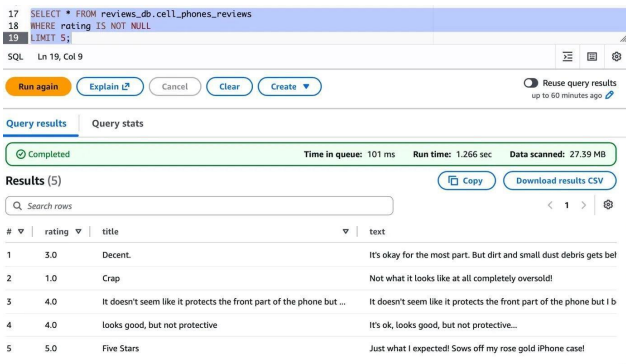


Fig. 19. Amazon Athena — SQL query on `reviews_db.cell_phones_reviews` confirming schema and sample records.

Fig. 20 shows the result of a `COUNT(*)` query on the same dataset. The output confirms that the table contains exactly 20,812,945 records, verifying that the full dataset has been successfully ingested, that no records were lost during loading or transformation, and that the dataset size matches the expected number of source records.

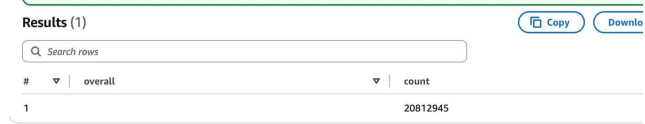


Fig. 20. Amazon Athena — `COUNT(*)` query confirming 20,812,945 total source records.

Fig. 21 shows the Cowork desktop interface with the amazon-reviews MCP connector enabled alongside Claude in Chrome. Cowork provides a task-oriented shell around the MCP server, allowing non-developer users to interact with fraud and sentiment tools through natural language without writing code. Fig. 22 shows the three tools exposed by the connector — `get_fraud_reviewers`, `get_top_products`, and `predict_review` — documented inline within the Cowork chat session.

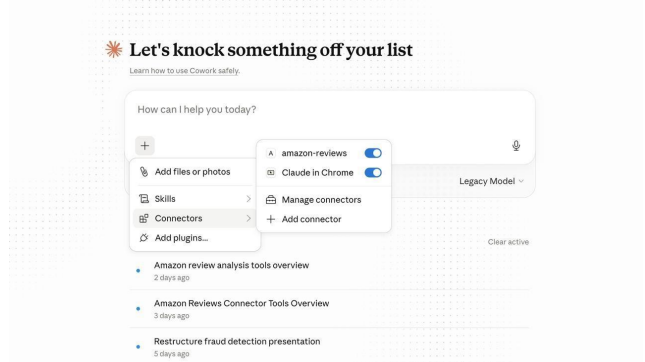


Fig. 21. Cowork desktop app — amazon-reviews MCP connector enabled alongside Claude in Chrome.

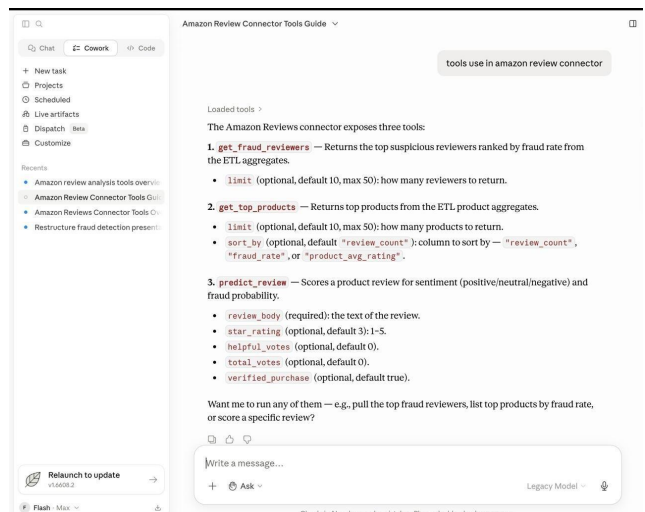


Fig. 22. Cowork — MCP server tools overview: get_fraud_reviewers, get_top_products, and predict_review.

Figures 23 and 24 show live tool calls made through the MCP connector. Fig. 23 displays the top 10 most-reviewed products using the get_top_products tool. Among them, product B01415QHYW has the highest number of reviews (42,644) along with a fraud rate of about 12%. Fig. 24 shows the top 5 products ranked by fraud rate. All of these products show a 100% fraud rate, but each has only 1–2 reviews — an important edge case: when a product has very few reviews, even a single flagged review can result in a 100% fraud rate. The system handles this correctly by explaining the situation in natural language.

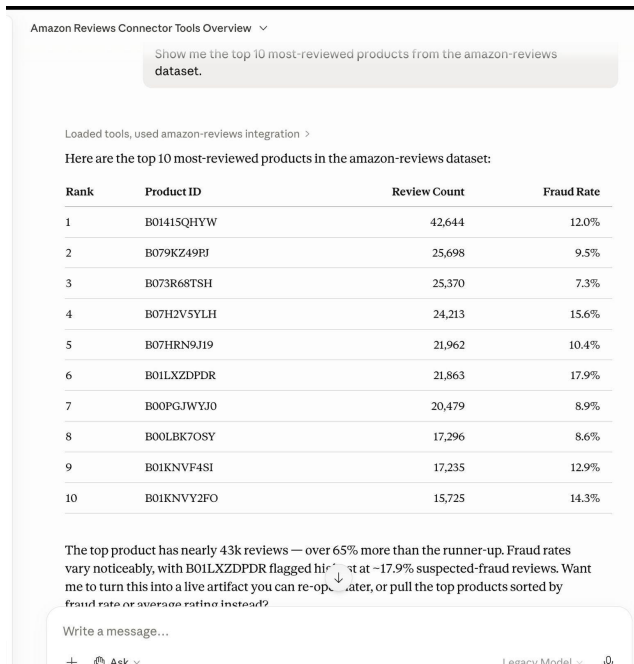


Fig. 23. Cowork — top 10 most-reviewed products by review count and fraud rate.

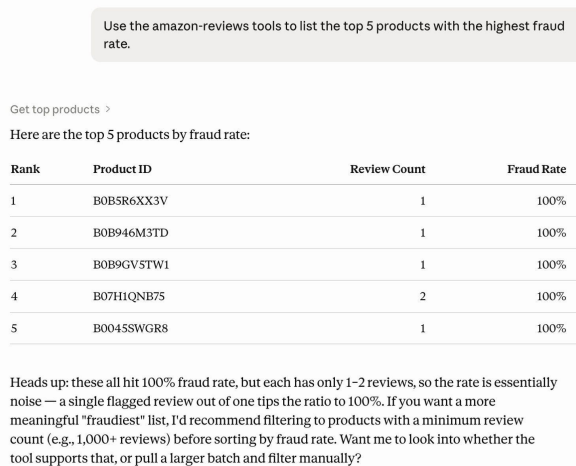


Fig. 24. Cowork — top 5 products by fraud rate (sorted by fraud_rate descending).

C. Autonomous LLM Auditor

The system includes an autonomous LLM-based review auditor that can analyze products and generate a risk assessment without manual intervention. This component is implemented in src/agents/review_auditor.py and uses the Ollama Llama 3.2 model. The auditor works in a multi-step loop, where the LLM can call different tools, observe the results, and decide the next step. It can perform up to 8 iterations, but usually reaches a conclusion much earlier. The available tools include get_product_aggregate (product-level summary), get_product_reviews (recent reviews), get_top_fraud_reviewers (suspicious reviewers), and score_review (individual review scoring). Based on the information gathered, the LLM generates a structured risk verdict for the product.

In testing, the auditor was applied to a sample product (ASIN: B01415QHYW), which has over 42,000 reviews and a fraud rate of about 12%. The system was able to identify it as a high-risk product after only a few tool calls, showing that the approach is both efficient and effective.

X. EXPERIMENTAL RESULTS

A. Full-Scale Spark Performance

To test how the system performs at scale, we ran a Spark aggregation job on the full dataset of 20.5 million reviews (about 3.8 GB after processing) on a single machine using a local Spark setup. The entire job, consisting of four stages, completed in about 30.8 seconds, showing that the pipeline is efficient even on limited hardware. One of the heavier stages, where data was grouped by reviewer, used 45 parallel tasks. Key aggregate results include an average product rating of 4.008, overall fraud rate of 3.57%, most-reviewed product B01415QHYW (42,644 reviews, ~12% fraud rate), and highest-fraud product with enough reviews B00BI9AKJI (41.3% fraud rate with 363 reviews).

B. Test Suite

To ensure the system works correctly, we created a set of 11 unit and integration tests. These tests check whether text cleaning is working properly, whether sentiment labels are correctly assigned from ratings, whether all required features are present during prediction, whether FastAPI endpoints respond correctly, and whether fraud explanations follow the expected format. All tests passed successfully in about 13.4 seconds, confirming that the pipeline is stable and reliable.

C. Runtime Summary

We measured how long each stage of the pipeline takes on a standard 16 GB laptop: data ingestion and schema conversion $\approx$ 3.5 minutes; Spark ETL processing $\approx$ 24 minutes; sentiment model training $\approx$ 52 minutes; fraud model training (SGD) $\approx$ 30 minutes; fraud model training (Logistic Regression) $\approx$ 3.5 hours; model diagnostics (threshold tuning, calibration, drift checks) $\approx$ 15 minutes. Overall, the complete pipeline — from

raw data to final results — takes around 6 hours to run end-to-end.

XI. DISCUSSION

A. Limitations

Although the system performs well, there are a few limitations. First, the dataset includes reviews from only a single product category (Cell Phones & Accessories). Because of this, category-level features do not add much value. Expanding the dataset to multiple categories would likely improve the model’s ability to generalize. Second, the dataset lacks information about unhelpful votes. Since total votes are equal to helpful votes, any feature based on their ratio becomes meaningless and cannot be used effectively. Another limitation is related to model convergence: the sentiment model (Logistic Regression with the saga solver) reached the maximum number of iterations before fully converging. While the model still performs well, the learned weights may not be fully optimized. A major challenge is the absence of true fraud labels: the model is trained and evaluated using weak labels based on heuristics rather than human-verified data, which means the reported performance may not fully reflect real-world accuracy. Finally, we originally planned to use Gradient Boosted Trees (GBT), but training them on such a large dataset was not practical on a single machine; we therefore switched to linear models, which are more scalable but may miss some complex patterns.

B. Future Work

There are several directions for improving the system. One immediate step is to integrate the tuned threshold (0.80) and calibrated probabilities directly into the serving layer for more accurate predictions in production. We also plan to expand the dataset to multiple product categories, which will allow category-based features to provide meaningful signals. Another important improvement is to create a small hand-labeled fraud dataset (around 1,000 reviews), which would provide a more reliable benchmark for evaluating model performance. From a system perspective, the entire pipeline can be packaged using Docker Compose, making it easier to deploy with a single command. We also aim to use the MLflow Model Registry to manage model versions and enable controlled promotion of models into production. Finally, replacing static behavioral features with a real-time feature store would allow the system to adapt dynamically as new data arrives.

XII. CONCLUSION

In this work, we presented a complete and reproducible pipeline for large-scale Amazon review analysis. The system processes over 20 million real reviews and performs both sentiment classification and fraud detection efficiently. The sentiment model achieves a weighted F1 score of 0.841, while the fraud model reaches a ROC-AUC of 0.845, all within approximately six hours on standard hardware. We ensured reliable performance by preventing data leakage, removing

rule-based features, and adding controlled label noise. Additionally, probability calibration significantly improved the quality of predicted scores. The system is not just theoretical — it includes multiple serving layers such as a REST API, Streamlit dashboard, MCP integration, an LLM-based auditor, and Kafka streaming. Together, these components make FraudLens a practical, end-to-end solution for large-scale review analysis.

Source code: https://github.com/Paruchuri-Rajesh/Large_Scale_Product-Review/tree/main

. REFERENCES

- [1] D. Kotzias, M. Denil, N. de Freitas, and P. Smyth, “From group to individual labels using deep features,” in Proc. ACM KDD, 2015, pp. 597–606.
- [2] M. Pontiki et al., “SemEval-2014 Task 4: Aspect based sentiment analysis,” in Proc. SemEval, 2014, pp. 27–35.
- [3] A. Mukherjee, B. Liu, and N. Glance, “Spotting fake reviewer groups in consumer reviews,” in Proc. WWW, 2012, pp. 191–200.
- [4] G. Fei, A. Mukherjee, B. Liu, M. Hsu, M. Castellanos, and R. Ghosh, “Exploiting burstiness in reviews for review spammer detection,” in Proc. AAAI ICWSM, 2013.
- [5] X. Meng et al., “MLlib: Machine learning in Apache Spark,” J. Mach. Learn. Res., vol. 17, pp. 1–7, 2016.
- [6] M. Zaharia et al., “Apache Spark: A unified engine for big data processing,” Commun. ACM, vol. 59, no. 11, pp. 56–65, 2016.
- [7] J. Ni, J. Li, and J. McAuley, “Justifying recommendations using distantly-labeled reviews and fine-grained aspects,” in Proc. EMNLP-IJCNLP, 2019, pp. 188–197. Dataset: McAuley-Lab/Amazon-Reviews-2023, Hugging Face.
- [8] S. J. Pan and Q. Yang, “A survey on transfer learning,” IEEE Trans. Knowl. Data Eng., vol. 22, no. 10, pp. 1345–1359, 2010.
- [9] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” Adv. Large Margin Classifiers, vol. 10, no. 3, pp. 61–74, 1999.